

## Process Management:

Everything that happens on a Linux server, a process is started. For that reason, process management is among the key skills that an administrator has to master.

There is Three Types of Process running in RHEL Linux:

1. Shell Jobs (commands started from the command line also called interactive jobs).
2. Daemons (processes that provide services. They normally are started when a computer is booted).
3. Kernel Threads (essential kernel threads that are started as real-time processes).

### HINT

Modern Linux kernels differentiate between essential kernel threads that are started as real-time processes and normal user processes. Increasing the priority of a user process will never be able to block out kernel threads or other processes that were started as real-time processes.

Using `ps` to get process information:

```
[root@server1 /]# ps aux | head
```

| USER | PID | %CPU | %MEM | VSZ   | RSS  | TTY | STAT | START | TIME | COMMAND                                                            |
|------|-----|------|------|-------|------|-----|------|-------|------|--------------------------------------------------------------------|
| root | 1   | 0.0  | 0.4  | 52984 | 4272 | ?   | Ss   | Feb05 | 0:03 | /usr/lib/systemd/systemd --switched-root --system --deserialize 23 |
| root | 2   | 0.0  | 0.0  | 0     | 0    | ?   | S    | Feb05 | 0:00 | [kthreadd]                                                         |
| root | 3   | 0.0  | 0.0  | 0     | 0    | ?   | S    | Feb05 | 0:00 | [ksoftirqd/0]                                                      |
| root | 5   | 0.0  | 0.0  | 0     | 0    | ?   | S<   | Feb05 | 0:00 | [kworker/0:0H]                                                     |
| root | 7   | 0.0  | 0.0  | 0     | 0    | ?   | S    | Feb05 | 0:00 | [migration/0]                                                      |
| root | 8   | 0.0  | 0.0  | 0     | 0    | ?   | S    | Feb05 | 0:00 | [rcu_bh]                                                           |
| root | 9   | 0.0  | 0.0  | 0     | 0    | ?   | S    | Feb05 | 0:00 | [rcuob/0]                                                          |
| root | 10  | 0.0  | 0.0  | 0     | 0    | ?   | S    | Feb05 | 0:00 | [rcuob/1]                                                          |
| root | 11  | 0.0  | 0.0  | 0     | 0    | ?   | S    | Feb05 | 0:00 | [rcuob/2]                                                          |

As you can see `ps` command has two flavors

BSD options written without hyphens and linux flavor.

## How to Differentiate Between the three types of shell Jobs ?!!

- a. Shell jobs is a command run from a shell
- b. Daemons is ended by d letter like (httpd – dhcpd – named) and of course the most important is the process with PID 1 which is system.
- c. Kernel threads between square brackets like the process with PID 2 which is kthreadd.

## Managing Shell Jobs

each and every command written on a shell is a shell job that means the shell is a parent and your command is the child ... (remember this)

Shell jobs occupying the terminal it was started from until it has finished its work. If this job will take longer time then you can change the default behavior from foreground to background.

For example if you type a command like :

#ls -l / it will run in foreground and displaying the content of / directory

But if you call application from the shell like firefox

#firefox

You will notice that the application is started but you are no longer able to run any command again on the shell screen so here you can decide to run the application in background to get your terminal back again.

## HOW ?!!!!

By end your command with &

#firefox&

#gedit&

**Ctrl+Z** to temporarily stop the job. This does not remove the job from memory; it just pauses the job so that it can be managed.

**Ctrl+C** This stops the current job and removes it from memory.

**Ctrl+D**, which sends the End Of File (EOF) character to the current job.

These abbreviations are so good for foreground processes, but to manage the background processes you can bring them back to foreground and run one of the previous abbreviations.

#jobs                   to display background processes

#fg %<index>       to bring to foreground

Or

Use the kill command (we will discuss later).

## Parent and Child Relationship:

As we mentioned earlier in this document the shell is the parent and the command is the child.

The relationship is:

1. For foreground process if you kill the parent it will kill the child.
2. For background process if you kill the parent the child is still running because its parent will be system.

Use **ps -ef** to See the Exact Command Used to Start Processes

```
[root@server2 ~]# ps -ef
```

| UID             | PID       | PPID     | C                                       | STIME                 | TTY  | TIME     | CMD                            |
|-----------------|-----------|----------|-----------------------------------------|-----------------------|------|----------|--------------------------------|
| root            | 874       | 1        | 0                                       | May14                 | ?    | 00:00:00 | /sbin/auditd -n                |
| root            | 885       | 874      | 0                                       | May14                 | ?    | 00:00:00 | /sbin/audispd                  |
| root            | 889       | 885      | 0                                       | May14                 | ?    | 00:00:00 | /usr/sbin/sedispach            |
| root            | 898       | 1        | 0                                       | May14                 | ?    | 00:00:00 | /usr/sbin/alsactl -s -n        |
| 19              | -c        | -E       | ALSA_CONFIG_PATH=/etc/alsa/alsactl.conf | --initfile=/lib/alsa/ |      |          |                                |
| root            | 899       | 1        | 0                                       | May14                 | ?    | 00:00:00 | /usr/sbin/bluetoothd -n        |
| root            | 900       | 1        | 0                                       | May14                 | ?    | 00:00:00 | /usr/bin/python -Es /          |
| usr/sbin        | firewalld | --nofork | --nospid                                |                       |      |          |                                |
| avahi           | 902       | 1        | 0                                       | May14                 | ?    | 00:00:00 | avahi-daemon: running          |
| [server2.local] |           |          |                                         |                       |      |          |                                |
| libstor+        | 903       | 1        | 0                                       | May14                 | ?    | 00:00:00 | /usr/bin/lsmc -d               |
| root            | 905       | 1        | 0                                       | May14                 | ?    | 00:02:14 | /usr/bin/vmtoolsd              |
| root            | 908       | 1        | 0                                       | May14                 | ?    | 00:00:00 | /usr/sbin/rsyslogd -n          |
| root            | 910       | 1        | 0                                       | May14                 | ?    | 00:00:00 | /usr/sbin/abrttd -d -s         |
| root            | 911       | 1        | 0                                       | May14                 | ?    | 00:00:00 | /usr/bin/abrt-watch-log        |
| -F              | BUG:      | WARNING: | at                                      | WARNING:              | CPU: | INFO:    | possible recursive locking det |
| avahi           | 912       | 902      | 0                                       | May14                 | ?    | 00:00:00 | avahi-daemon: chroot           |
| helper          |           |          |                                         |                       |      |          |                                |

## Sending Signals to Processes with kill, killall, and pkill

kill command or sending other signals to processes, it is good to know that Linux processes have a hierarchical relationship.

- The signal SIGTERM (15) is used to ask a process to stop. (default signal)
- The signal SIGKILL (9) is used to force a process to stop.
- The SIGHUP (1) signal is used to hang up a process. The effect is that the process will reread its configuration files, which makes this a useful signal to use after making modifications to a process configuration file.

#kill <PID>        each process has it's ID

#kill <PPID>

How we know the PID or the PPID ?!! from ps aux or ps -ef .

Or pgrep or pidof commands but the recommended is ps -ef as we mention in process management lecture.

Kill with pkill <PROCESS> is not recommended.

## Using top to Manage Processes

### Priorities ( 0 Normal , -20 highest , 19 lowest)

```
root@server1:~
```

| File | Edit | View | Search | Terminal | Help |
|------|------|------|--------|----------|------|
|------|------|------|--------|----------|------|

```
top - 16:02:38 up 16:05, 3 users, load average: 2.13, 2.08, 1.92
Tasks: 392 total, 4 running, 387 sleeping, 1 stopped, 0 zombie
%Cpu(s): 10.4 us, 79.6 sy, 10.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem: 1010880 total, 934768 used, 76112 free, 40 buffers
KiB Swap: 1048572 total, 296580 used, 751992 free. 450752 cached Mem
```

| PID   | USER | PR | NI  | VIRT    | RES    | SHR   | S | %CPU  | %MEM | TIME+    | COMMAND      |
|-------|------|----|-----|---------|--------|-------|---|-------|------|----------|--------------|
| 17449 | root | 20 | 0   | 107920  | 624    | 532   | R | 100.0 | 0.1  | 92:52.18 | dd           |
| 18666 | root | 25 | 5   | 107920  | 624    | 532   | R | 98.0  | 0.1  | 31:58.40 | dd           |
| 3300  | user | 20 | 0   | 1614116 | 151584 | 18028 | S | 1.0   | 15.0 | 1:34.06  | gnome-shell  |
| 19150 | root | 20 | 0   | 123792  | 1836   | 1152  | S | 0.7   | 0.2  | 0:00.09  | top          |
| 1066  | root | 20 | 0   | 197196  | 17740  | 2036  | S | 0.3   | 1.8  | 0:05.29  | Xorg         |
| 1     | root | 20 | 0   | 52984   | 4272   | 2176  | S | 0.0   | 0.4  | 0:04.04  | systemd      |
| 2     | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.08  | kthreadd     |
| 3     | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.51  | ksoftirqd/0  |
| 5     | root | 0  | -20 | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | kworker/0:0H |
| 7     | root | rt | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.18  | migration/0  |
| 8     | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcu_bh       |
| 9     | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/0      |
| 10    | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/1      |
| 11    | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/2      |
| 12    | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/3      |
| 13    | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/4      |
| 14    | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/5      |
| 15    | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/6      |
| 16    | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/7      |
| 17    | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/8      |
| 18    | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/9      |
| 19    | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/10     |
| 20    | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/11     |
| 21    | root | 20 | 0   | 0       | 0      | 0     | S | 0.0   | 0.0  | 0:00.00  | rcuob/12     |

. From top, type **k**. top will

then prompt for the PID of the process you want to send a signal to. By default, the most active process is selected. After you enter the PID, top asks which signal you want to send. By default, signal 15 for SIGTERM is used. However, if you want to insist a bit more, you can type **9** for SIGKILL. Now press **Enter** to terminate the process.

To renice a running process from top, type **r**. You are first prompted for the PID of the process you want to renice. After entering the PID, you are prompted for the nice value you want to use. Enter a positive value to increase process priority or a negative value to decrease process priority.

Using nice and renice commands

```
#nice -n -15 firefox
```

```
#renice -n 9 8875
```

لمعرفة ناتج تنفيذ اي امر من الاوامر السابقة يرجى مراجعة المحاضرة

*Eng. Muhammad Adel*